

Table of Contents

Studium — Frontend and Interaction Specification

Subsystem 4 of 7. Version 1.0. Status: build-ready.

Written against data layer specification v1.1, agent runtime specification v1.0, and retrieval specification v1.0. The nine pending items across those three specs (V1–V3, V13, V14, V15 retention, R4 exchange_index, and the two retrieval additions chunk_type and tsvector_text) are noted but do not affect the frontend contract. The seven ratified subsystem-2 v1.1 items are similarly noted; frontend consumes agent output through the SSE contract established in subsystem 2 §20, which is unchanged.

1. Overview

This document specifies the frontend for Studium: the Next.js application that consumes agent streams, renders lectures with hoverable citations, exposes tutorial interaction primitives, handles the “raise your hand” interruption, and meets WCAG 2.2 Level AA. The frontend is what the learner actually sees and touches; everything the previous three subsystems produce becomes worth building only insofar as this layer surfaces it well.

A senior engineer with data layer v1.1, agent runtime v1.0, retrieval v1.0, and this document should be able to build a working web frontend that lets a learner open a session, receive streaming lectures, ask questions mid-stream, invoke tutorial primitives, work through practice problems, review their confusion journal, and close a session — all with keyboard operability, screen-reader announcements, and a visual language that respects the seriousness of what the learner is doing.

The frontend is the surface on which the aesthetic direction from the product plan becomes concrete. Where subsystems 1–3 could remain visually agnostic (a database schema has no font), this one cannot. §16 makes the aesthetic choices explicit and answerable: what typography, what palette, what motion, what voice — chosen against a small set of reference points and defensible against alternatives.

2. Scope and non-goals

In scope.

- The full Next.js application: routes, layouts, server components, client components, streaming, state management.
- The SSE consumer: how streams from subsystem 2 are consumed, buffered, rendered, and interrupted.

- The five MVP surfaces: the desk (session start / dashboard), the classroom (lecture and tutorial rendering), the bench (lab / practice), the journal (confusion journal), and session close (summary and next-steps).
- The tutorial interaction primitives UI: eight primitives with button, keyboard, and command-palette access.
- The “raise your hand” interruption UX: precisely how the learner interrupts, what they see while it resolves, and how the transition back to the lecture is handled.
- Citation rendering: how [P1] markers become hover cards with source metadata, resolved via subsystem 3’s endpoint.
- Comprehension checks and practice questions: how they present, how attempts work, how feedback lands.
- Accessibility: WCAG 2.2 Level AA compliance, including keyboard operability, focus management, screen reader support, reduced motion, and color contrast.
- State management: server state via TanStack Query, client state via Zustand, URL state for routing.
- Error handling and degradation in the UI: what the learner sees when the backend degrades, when the network drops, when a budget cap trips.
- The aesthetic and typographic system: type choice, palette, spacing, motion, tone of copy.
- Testing strategy: component tests, accessibility tests, end-to-end tests.

Explicitly out of scope.

- The four MVP-deferred surfaces: the concept graph map (subsystem 4 v1.1 candidate once the concept graph module ships), the review cycle UI (subsystem 4 v1.1 once the review module ships), the multi-voice library UI, and the summative assessment UI. Anticipated architecturally so their addition is additive.
- Mobile clients. Web-only for MVP per the earlier decision. Responsive design that works on tablets is a happy accident, not a target; phone screens are explicitly out of scope until a dedicated mobile spec exists.
- Voice input and output. Text-first for MVP. The UI is designed so a voice layer can be added later without restructuring — every interactive element has a text equivalent — but voice-specific concerns (barge-in latency, ASR partials, microphone permission) live in a subsequent voice spec.
- Native desktop apps, PWA installation, offline mode. Web app only, online only.
- Multiplayer, sharing, social features. Studium is a single-learner product at MVP.
- Payment and subscription UI. No commerce at MVP (no marketing tool per your earlier decision).
- Admin surfaces beyond the content review queue viewer. Full admin functionality (user management, system dashboards) is deferred until it’s needed.

3. Design principles

The interface is the learner’s environment for serious work. This is the operative frame for every design decision. Fonts, spacing, motion, copy tone — all read as though

they belong in a well-kept study, not a consumer app. The visual language of “engagement” (bright colors, animated mascots, streaks and badges) is optimized for a problem this product does not have and would actively undermine. See §16 for how this becomes concrete.

Every interactive element is keyboard-operable. No mouse-only interactions. This is a WCAG 2.2 AA requirement (§13) and it is also a productivity property that experienced learners appreciate. Focus indicators are visible and correct; tab order matches visual order; keyboard shortcuts exist for the operations learners will invoke frequently, and are discoverable via a ? help overlay.

Streaming is a first-class rendering mode, not a UX afterthought. A learner reading a lecture as it streams should have the same reading experience they would if the whole segment appeared instantly — the same typography, the same layout stability, the same ability to select and copy text as it lands. Chunks arriving don’t cause visible reflow beyond adding new characters; citations render as they appear rather than after the stream completes.

Interruption is a preserved control, not a hidden power-user feature. The “raise your hand” mechanic is discoverable, unambiguous, and always available during streaming. A learner should never wonder whether they can interrupt or how; the affordance is visible and its behavior is predictable. This is spec’d in §8 with more care than any other single interaction.

Accessibility is designed in, not retrofitted. The dev writing components consults §13’s WCAG checklist as they build, not after. Component library choices (§4 recommends Radix) provide accessibility primitives that would be expensive to hand-roll. axe-core runs against every component in CI. Reference to §13’s specific requirements appears throughout the rest of this document where they intersect a design decision.

Server state and client state are architecturally separated. Data that lives in the database (user, sessions, journal, mastery) is server state and flows through TanStack Query with defined cache invalidation. Data that only matters to the UI (current stream buffer, whether the interrupt button is armed, dark mode preference) is client state and lives in Zustand or component-local useState. No server data is mirrored into client state; no client state is persisted to the backend. This is the boundary that prevents entire classes of consistency bugs.

Optimistic updates are cheap and rollback is designed. Where a mutation is nearly-always-successful (marking a journal entry as resolved, submitting an answer, invoking a primitive), the UI updates immediately and rolls back on error with clear notification. Where a mutation has real chance of failure or is expensive (starting a session, submitting an assessment), the UI shows a pending state and waits. The line between these cases is explicit in each surface’s spec.

Degradation is legible. When the backend degrades — budget cap trips, model API rate limits, network drops — the learner sees a coherent, calm message that names what happened and what to do next. Never a raw error, never a spinner without explanation,

never a silent failure. The degradation copy library from subsystem 2 §21 lives here in `studium-web/copy/degradation.ts`.

Content is quiet; controls are unobtrusive. The reading experience dominates the visual field. Controls (buttons, menus, chrome) recede when not in use and appear when relevant. This is not asceticism — a lecture that the learner is reading benefits from having nothing else on screen competing for attention, and returning controls when the learner's attention moves to interaction is the reciprocal courtesy.

4. Technology stack

Framework. Next.js 15+ with the App Router. Server Components by default; Client Components ("use client") where interactivity is required. React 19+.

Language. TypeScript 5+. `strict: true, noUncheckedIndexedAccess: true`. No any types in application code; escape-hatch unknown with narrowing where truly necessary.

Styling. Tailwind CSS 4+ with a small custom theme (fonts, palette, spacing scale — see §16). No CSS-in-JS. No component-library CSS imports beyond Radix's unstyled primitives.

Component library. Radix UI Primitives for accessibility-critical interactive components (Dialog, Popover, Tooltip, DropdownMenu, Toolbar, Toast, Tabs). Radix's unstyled primitives get all the ARIA attributes, focus management, and keyboard handling right; Tailwind provides the visual layer on top. Rolling these components from scratch would be expensive and would fail accessibility in specific ways WCAG 2.2 AA catches.

Server state. TanStack Query 5+. All server data flows through query keys with defined cache lifetimes. Mutations use `useMutation` with optimistic updates where §3's principle applies.

Client state. Zustand 5+ for global client state (current session state, UI preferences, stream buffer). Component-local `useState` for local concerns.

Schema validation. Zod for runtime validation of API responses. Every API response is validated at the boundary; internal code trusts validated types.

Streaming. Native `EventSource` for SSE consumption. No third-party library — the browser API is sufficient for the use case and reduces dependency surface.

Icons. Lucide React. Consistent stroke weight, minimal visual noise, works at small sizes for keyboard-shortcut annotations.

Typography. Two families: a body serif (see §16) and a UI sans (see §16). Both loaded via `next/font/google` for self-hosted delivery.

Math and code. KaTeX for math rendering (inline $\dots$ and display $\dots$). Shiki for code syntax highlighting via `shiki-transformers`. Both used server-side where possible for stable layouts.

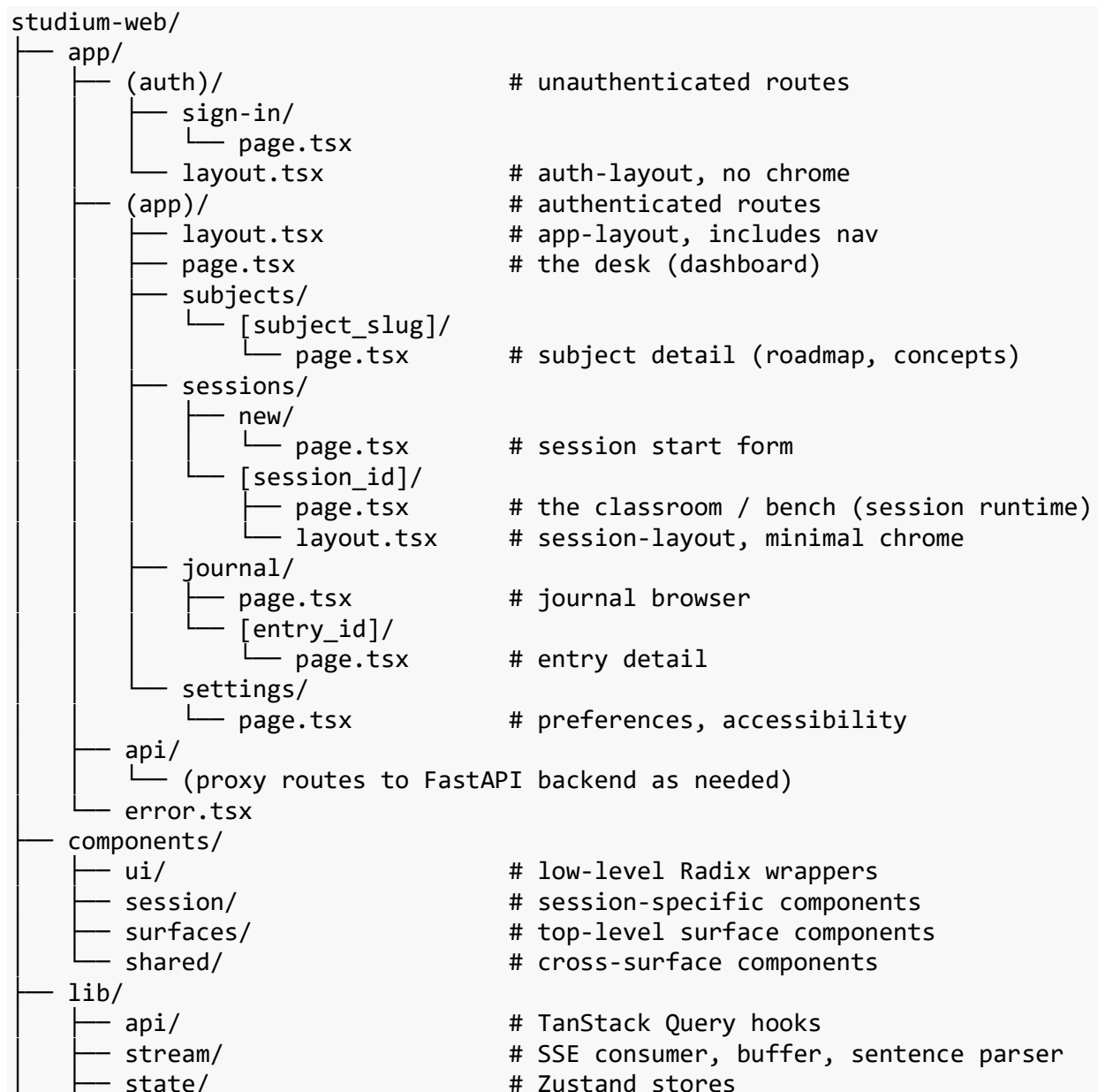
Testing. Vitest for unit and component tests. Testing Library for React component tests. axe-core via vitest-axe for accessibility assertions. Playwright for end-to-end tests including the paid tier that exercises the real backend.

Build and runtime. Node.js 22 LTS. Deployed on Fly.io Toronto region alongside the FastAPI backend (Infrastructure spec, subsystem 7). Next.js production build; no server-side rendering fallback needed since the app requires authentication.

5. Application architecture

5.1 Route structure

Next.js App Router with route groups for organizational clarity.



```

└─ a11y/                # accessibility utilities
└─ copy/                # copy strings (including degradation)
└─ styles/
└─ globals.css          # Tailwind directives, CSS variables

```

The (auth) route group excludes app chrome (navigation, footer). The (app) group includes it. The session route (sessions/[session_id]) has its own layout that hides most chrome to give the session content the visual field.

5.2 Server vs client components

Default is Server Component. Client Components exist for these reasons:

- **Interactivity** — anything with onClick, onChange, onSubmit, keyboard handlers.
- **State** — anything using useState, useReducer, Zustand.
- **Effects** — anything using useEffect, useMemo, useCallback.
- **Streaming consumption** — the SSE consumer is a Client Component.
- **Third-party libraries requiring browser APIs** — chart libraries, editors, etc. (none of these in MVP, but noted for later).

The pattern for a typical surface: Server Component at the route (fetches initial data), delegates interactive regions to Client Components. Streams (from server) plus client-side navigation, not server-side rendering everything.

5.3 Data fetching

Server Components fetch via async functions calling the FastAPI backend. Client Components use TanStack Query hooks that call the same endpoints. The two paths share Zod schemas defined in lib/api/schemas.ts so the validation is one source of truth.

Query key convention. Nested tuples matching resource paths:

```

['user', userId]
['user', userId, 'sessions']
['session', sessionId]
['session', sessionId, 'turns']
['journal', userId, { status: 'open' }]
['artifact', artifactId, 'citations']

```

Cache lifetime. Server state has explicit staleTime per query type:

Data	staleTime	Rationale
User profile	5 minutes	Rarely changes
Subject list	5 minutes	Changes only on enrollment
Concept mastery snapshot	30 seconds	Updates during sessions
Journal entries	30 seconds	Same
Session turns	0 (always refetch on view)	Real-time relevance
Artifact citations	1 hour	Immutable per artifact

Optimistic mutations. Marking a journal entry resolved, submitting a practice answer, invoking a primitive — these show the effect immediately, roll back on error with a toast. Starting a session, closing a session, submitting a summative assessment — these show a pending state and wait for the server.

6. The five MVP surfaces

Each surface is a top-level route with a specific purpose. The four MVP-deferred surfaces (concept graph map, review cycle UI, multi-voice library, summative assessment) are noted where their addition would extend an existing surface but not built.

6.1 The desk (/)

The default view when the learner opens the app. Its purpose is to answer three questions at a glance: what did I do last, what should I do now, and what is still open.

Layout. Two columns on wide viewports, single column below 900px:

- **Left column (60%):** Continue-session card, recent-sessions list, current-syllabus preview.
- **Right column (40%):** Open journal entries (top 5), review queue count (deferred but reserves layout space), mastery summary.

Continue-session card. Shows the most recent open or recently-closed session with a “Continue” or “Start next session” primary action. If no session exists (first-time user), replaces with a “Begin studying” prompt.

Recent sessions list. Last 5 closed sessions with mode, duration, concepts covered (from `session_summaries.concepts_touched`), and a link to view the summary. Passive list; no bulk operations.

Current syllabus preview. Reads from `learner_subjects.syllabus_plan`, shows the next 3 concepts. Not clickable in MVP (the concept graph map, which would let the learner jump ahead, is deferred).

Open journal entries. Reads from `journal_entries` WHERE `status` IN ('open', 'partial') sorted by `last_touched_at` DESC, limit 5. Each entry shows summary, concept name, age. Click to open entry detail.

Mastery summary. A small visual (bar chart or heatmap) showing overall subject mastery. Not the concept graph — a summary reduction of it. Placeholder for the map view when it ships.

6.2 The classroom (/sessions/[session_id] when mode is lecture or tutorial)

The lecture and tutorial surface. This is where the learner spends most of their time.

Layout. Single centered column, ~65 characters wide for reading comfort. Two persistent chrome elements:

- **Top:** minimal session header — subject name, current concept name, session progress indicator, close-session button (with confirmation).
- **Bottom-right:** floating action group — the interrupt button, the primitive command palette trigger, session timer, cost indicator (if the learner has enabled it in preferences).

Body. Streams lecture text or tutorial dialogue into the reading area. Text renders as it arrives with citation markers ([P3]) becoming hover cards in place. Math and code blocks render with proper typography as they land.

Comprehension check inline. When the Lecturer generates a check (subsystem 2 §10), it appears as an inset card within the reading flow, not as a modal. Answer input below the check text; feedback below the answer after submission.

Tutorial exchange. Tutor turns render as reading content, styled subtly differently from lecture segments (indent, distinct type treatment). Learner input area appears at the bottom of the reading column, expanding as needed.

Deferred elements this surface anticipates: the concept graph map (would appear in a right sidebar), the multi-voice library switcher (would live in the header near concept name).

6.3 The bench (/sessions/[session_id] when mode is lab)

The lab surface. Practice problems, worked examples, and executable work.

Layout. Two columns on wide viewports: problem statement and workspace on left, feedback and hints on right. Single column below 900px, with workspace above feedback.

Problem area. Shows the practice problem generated by the Lecturer or selected by the Curator. Includes any setup, constraints, and the specific ask.

Workspace. Where the learner produces their work. For text answers: a plain textarea with reasonable defaults (monospace for code-adjacent subjects, serif for prose). For code (deferred but anticipated): a Monaco editor with language detection.

Feedback area. Where the Evaluator's response appears after submission. Verdict (correct / partial / incorrect) with both color and icon (never color-alone, per §13). Rubric-based breakdown for assessment attempts; single-criterion verdict for practice checks. Model answer appears after the last allowed attempt.

Hint mechanism. Hints are collapsed by default; expanding one costs nothing (no penalty) but is recorded to the trace so the reviewer can see when hints were needed.

6.4 The journal (/journal)

The confusion journal browser. All open, partial, and resolved entries across the learner's subjects.

Layout. Left sidebar with filters (status, subject, concept), main area with entry list.

Filters.

- Status: All / Open / Partial / Resolved / Archived. Default: Open + Partial.
- Subject: multi-select. Default: all.
- Concept: multi-select within the currently-filtered subjects.
- Date range: default last 30 days.

Entry list. Each entry shows the summary, concept name, status pill (color + icon), last-touched date, and a preview of the hypothesis if it exists (with a “why this was flagged” tooltip explaining that the hypothesis is the system’s inference, not the learner’s own words).

Entry detail (/journal/[entry_id]). The full entry with:

- The summary (editable by the learner).
- The system-generated hypothesis (read-only, with the “system inference” framing spec’d in subsystem 2 §12).
- The learner’s own note (editable, free-form).
- History timeline: created, revisited, addressed, resolved events with session links.
- Actions: mark resolved, mark partial, reopen, archive, delete.

Search. Full-text search across entry summaries and learner notes. Not fuzzy search — a substring match is sufficient at MVP scale. Uses pg_trgm GIN index; deferred until entry count exceeds ~100 per learner.

6.5 Session close

Not a route — a modal that appears when a session ends. Purpose: close the loop clearly, show what was covered, name what’s next.

Contents.

- Session summary text (from `session_summaries.summary`, generated by the Curator at close per subsystem 2 §16).
- Concepts touched (with mastery deltas: “beta-reduction: 0.65 → 0.82”).
- Open threads carried forward: any items the summary flagged for next time.
- What’s next: the Curator’s suggested next focus concept, with a “Start next session” primary action.
- Duration and cost (if learner has cost display enabled).

Interaction. Dismissible with Escape or an explicit “Done” button. Cannot be dismissed by clicking outside — the summary is the closing beat of a session and merits explicit acknowledgment. Once dismissed, the learner returns to the desk.

7. Session lifecycle in the UI

The lifecycle mirrors subsystem 2 §16 with UI-specific concerns added.

7.1 Opening

Trigger. Learner clicks “Continue” or “Start next session” from the desk, or navigates to /sessions/new.

UI flow.

1. Session start form: mode selector (lecture / tutorial / lab / review — review deferred), target duration slider (defaults to learner’s preferences.default_session_minutes, typically 90), focus concept (defaults to learner_subjects.current_focus_concept_id).
2. Submit → POST to backend, receive session_id.
3. Navigate to /sessions/[session_id].
4. Classroom or bench renders. If prior context exists (per subsystem 2 §16), the retrieval check appears first as an inline card in the classroom before the lecture starts.
5. Session content begins streaming.

Budget pre-flight. If the backend refuses session start due to budget cap (subsystem 2 §19), the UI shows the BudgetExceededError copy with reset time in the learner’s timezone. No navigation to the session route; the learner stays on the start form with the error prominently placed.

Loading state. Between form submit and stream start, show a calm loading state (“Preparing your session...”) — not a spinner alone. On slow starts (>2 seconds), append a note that opening a session includes context assembly and can take a moment.

7.2 Body

Streaming. The stream begins; text renders as it arrives (see §8). Interruption is possible from the moment the first token lands.

Turn transitions. When a lecture segment completes, the surface transitions to the next segment or to a comprehension check without page navigation — the classroom stays mounted, the content changes. This preserves scroll position within a segment while making segment boundaries clear.

Comprehension checks. Rendered inline (§6.2). Two-attempt cycle per subsystem 2 §10:

1. Learner submits first answer.
2. Evaluator grades; if correct, feedback + next segment. If incorrect, hint appears, second attempt allowed.
3. Second incorrect → model answer revealed, brief acknowledgment, next segment.

Feedback appears with a 300ms delay after the grade completes, so the transition doesn’t feel abrupt.

Primitive invocation. Learner presses / or clicks the command palette trigger, selects a primitive, provides input if needed. The current stream (if any) is not interrupted for

primitive invocations that don't require it (`why_does_this_matter` doesn't stop the flow; `explain_differently` does).

7.3 Closing

Trigger. Learner clicks close button, session hits `target_duration_minutes + 15` idle timeout, or budget hard cap trips.

Close flow.

1. Confirm dialog: "Close this session? Your progress is saved and a summary will be generated." Two options: Close, Cancel. On idle timeout or budget trip, no confirm — the flow proceeds directly.
2. Show "Wrapping up your session..." state. The Curator's summary generation is an LLM call (subsystem 2 §16), takes 5–15 seconds.
3. Summary modal appears (§6.5).
4. On dismiss, navigate to /.

7.4 Cross-session persistence

The retrieval check at session start (subsystem 2 §16, backed by data layer §6.11) uses the prior session's summary to generate 2–3 quick prompts that verify the material has stuck.

UI treatment. Renders as the first inline card in the classroom before any new lecture content. Learner sees the prompts, answers each, receives immediate verdict. Results feed the mastery model but the UI framing is low-stakes ("A quick check on last time before we continue"). The results are not shown as a "score" — a summary sentence appears at the bottom ("You're solid on beta-reduction; we'll spend a moment on alpha-conversion at the start of today's material") and the session proceeds.

8. Streaming and interruption UX

This is the load-bearing interaction section. Two things must work correctly: streaming rendering that respects reading flow, and interruption that is discoverable, predictable, and non-destructive.

8.1 SSE consumption

The stream consumer is a Client Component hook: `useSessionStream(sessionId)`. It:

1. Opens an EventSource connection to `/api/session/{sessionId}/stream`.
2. Parses each data: `{...}\n\n` event into a `StreamChunk` (`kinds: text, tool_effect, trace, end`, per subsystem 2 §6).
3. Dispatches by kind:
 - `text` → appends to the current message buffer, triggers re-render.
 - `tool_effect` → not displayed; used for internal state updates (rare — usually applied server-side).
 - `trace` → not displayed; logged to the browser console in dev.

- end → closes the buffer, marks the turn complete, awaits next stream.

Reconnection. If the connection drops (network hiccup, backend restart), EventSource reconnects automatically. On reconnection, the frontend sends a resume request with the last received `turn_index`; the backend either resumes from that point or acknowledges the stream is complete.

Buffering. Text chunks arrive at variable rates. The renderer buffers by a soft ~50ms window — chunks arriving within the window batch into a single render pass. This prevents excessive re-renders on high-frequency streaming without introducing visible lag.

8.2 Rendering as the stream arrives

Markdown rendering. Text is rendered as Markdown continuously. The renderer must handle incomplete Markdown gracefully — an open code fence at the moment of render should not visually break the layout. The pattern: render text using a Markdown parser that tolerates partial input (react-markdown with the remark-gfm plugin handles this well); wrap trailing incomplete blocks in a “still generating” style that resolves when the block closes.

Math rendering. $...$ and $...$ blocks render via KaTeX once the closing delimiter arrives. Incomplete math (opening $...$ with no closing) renders as literal text until the closing delimiter, at which point it re-renders as math. Visible flash accepted — the alternative (waiting for closing to render anything) is worse.

Code blocks. Similar to math. Triple-backtick fences render as code once closed; incomplete fences render as literal text.

Citation markers. [P3] markers render as hoverable superscripts as soon as they appear in the text. The citation resolution is fetched lazily on first hover — the marker is visible immediately, the hover card appears on hover.

Scroll behavior. As new text arrives, the viewport does not auto-scroll — the learner’s current reading position is preserved. A “New content” indicator appears at the bottom of the viewport when new text has arrived below the fold, tappable to scroll to it. Auto-scroll is a legitimate preference for some learners; it’s toggleable in settings (default off).

8.3 The “raise your hand” interruption

The affordance:

- **Visible button.** A hand-raise button lives in the bottom-right floating action group during any streaming state. Icon (Hand from Lucide), label “Interrupt” on hover, keyboard shortcut annotation [Space]. Button state: normal when idle, filled when a stream is active and interruption is possible, disabled when in a state where interruption doesn’t apply (comprehension check evaluation, primitive resolution, closing).

- **Keyboard shortcut.** Space bar interrupts when the focus is anywhere in the document (not inside a text input — inside an input, Space types a space). This matches the video-player convention learners are already familiar with.
- **Typing.** Beginning to type in the always-available message input at the bottom of the classroom is also treated as an interrupt intent. This is lower-friction for learners already at the keyboard.

The behavior on interrupt:

1. Client sends POST to `/api/session/{sessionId}/interrupt`. Optimistic UI: the button visually depresses, aria-live announces “Requesting interrupt”.
2. Server acknowledges with 202 Accepted; the streaming loop signals sentence-boundary detection.
3. Stream continues arriving until the current sentence completes (per subsystem 2 §20’s “finish current sentence” model).
4. Sentence completes; a subtle visual marker appears (thin horizontal rule) showing where the lecture paused. Aria-live announces “Ready for your question”.
5. Message input receives focus. Placeholder changes to “Your question...”. Send button appears.
6. Learner types and submits. New stream begins from the Tutor. Aria-live announces “Tutor is answering”.
7. When the Tutor’s answer completes, a resumption card appears with two options: “Continue lecture” or “Ask another question”. Continue triggers the resume: brief recap from the interruption point, then the lecture stream picks up.

Cancellation. If the learner presses Escape before typing anything, the interrupt is cancelled — no new stream from the Tutor, the interrupted lecture resumes with a two-sentence recap. Aria-live announces “Resuming lecture”.

Consecutive interrupts. A learner can interrupt the Tutor’s answer, chain into a follow-up question, and continue that thread for as long as they want. Each interrupt is one turn in `session_turns`; the Orchestrator’s state machine handles the transitions (subsystem 2 §7).

The “escalate to office hours” option. If a thread of interrupts is running long (heuristic: 3+ Tutor turns without resuming the lecture, or 10+ minutes elapsed since the interrupt), a subtle prompt appears offering to escalate to a full office hours session. Learner can accept (session mode changes) or dismiss.

8.4 Interruption edge cases

- **Interrupt during a comprehension check.** Check acknowledges the interrupt but does not clear the check — the check’s answer input remains visible above the interrupt input. Learner can answer the interrupt first and return to the check, or dismiss the interrupt and return.
- **Interrupt during a primitive.** Some primitives (`explain_differently`, `show_worked_example`) themselves involve a stream. Interrupt during these behaves

the same as interrupt during a lecture stream — sentence completes, Tutor takes over.

- **Interrupt during session close.** Close cannot be interrupted; the close button is disabled during the summary generation. This is deliberate — the summary is the record of the session and should complete cleanly.

9. Tutorial primitives UI

The eight primitives from subsystem 2 §15 are surfaced via three access paths: the command palette (keyboard-first), contextual buttons (mouse-first), and voice commands (deferred).

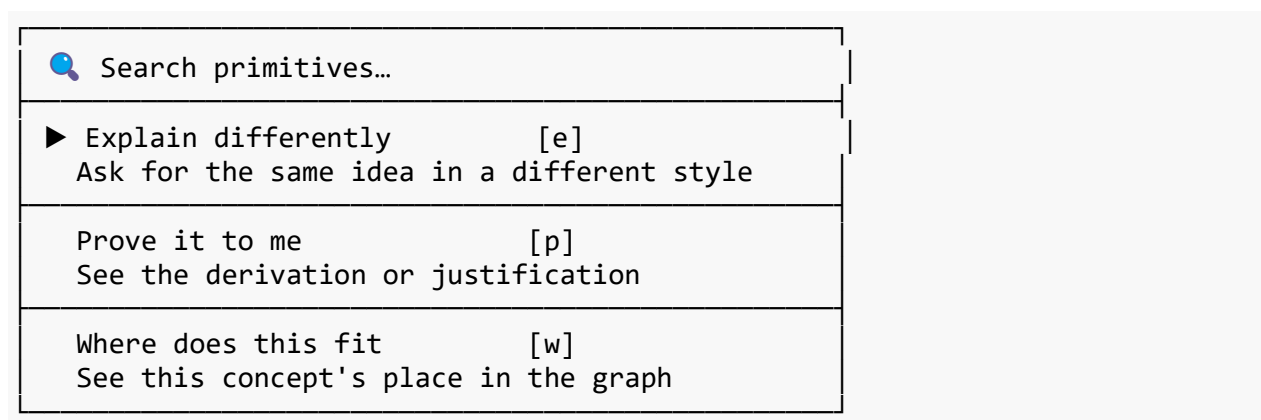
9.1 Command palette

Activated by / (matching the convention from Slack, Notion, GitHub) or Cmd+K (macOS) / Ctrl+K (other). Opens an overlay with a text input and a filterable list of primitive commands.

Behavior.

- Typing filters the list. Fuzzy match on primitive name and short description.
- Enter selects the highlighted primitive.
- Escape closes without selecting.
- Focus is trapped in the palette while open (Radix Dialog handles this).
- Screen reader announces the palette opening and the number of matching options as the learner types.

Layout.



Each primitive has a single-letter shortcut visible on the right. Learners who use primitives often internalize the shortcuts and eventually invoke them without opening the palette.

9.2 Contextual buttons

The bottom-right floating action group during a session includes 2–3 contextually-relevant primitive buttons. Which ones show depends on session state:

State	Buttons
LECTURING	Explain differently, Show worked example, I'm lost
TUTORIAL	Prove it to me, Where does this fit, I'm lost
LAB	Show worked example, I'm lost
PAUSED_FOR_QUESTION	(no primitive buttons — the question flow is the primitive)

The full palette is always accessible via the command palette; contextual buttons are a mouse-user convenience for the primitives most likely to be wanted in each state.

9.3 Primitive-specific UX

Each primitive has small variations in how it's invoked and how its output appears.

explain_differently. Two-step: Tutor first asks what didn't land ("What about the current explanation isn't working for you?"); learner types a brief response; Lecturer regenerates in a new stance. Renders as a Tutor exchange followed by a Lecturer segment.

prove_it_to_me. Tutor first asks "What would you expect the argument to look like?" — trying to elicit the derivation from the learner. If learner can't start, Tutor walks it. Renders as a Tutor exchange.

where_does_this_fit. Tutor renders the concept's place in the graph as prose ("This concept depends on X and Y, and it's a prerequisite for Z. It sits in the same family as A, which you'll see later"). MVP has no visual graph — the map view is deferred — so this is text-only. When the map ships (v1.1), this primitive additionally highlights the concept in the map view.

vocabulary_check. Tutor asks whether the confusion is about a specific term or about the underlying idea. If term: paraphrase + glossary link. If idea: opens a Socratic sub-thread.

show_worked_example. Lecturer produces a fully worked example, step-by-step. Streams as a lecture segment.

let_me_try_one. Curator selects a practice problem at the current mastery level; session transitions to LAB mode; problem renders on the bench.

why_does_this_matter. Tutor explains what the concept enables downstream. Renders as a short Tutor exchange.

im_lost. Tutor asks what the last thing that made sense was; Curator resets the session focus to the last high-mastery concept in the neighborhood; brief bridge delivered. Renders as a Tutor exchange followed by (usually) a Lecturer re-orientation.

10. Citation rendering

Citations use the resolution endpoint from retrieval subsystem §12.

10.1 Inline marker rendering

The regex `\[P\d+(-P\d+)?\]` matches citation markers in streaming text. Each match is replaced with a `<Citation>` component that renders as a superscript link.

Visual. Superscript number matching the marker, in the body serif at 80% size, with a subtle underline on hover. `[P3]` renders as a small 3 above the baseline.

Ranges. `[P3-P5]` renders as 3-5 in the same style, resolving to three citations on click.

Accessibility. Each citation has an `aria-label` describing the source (“Citation 3: Michaelson, page 42”) so screen readers announce meaningful context, not just a number.

10.2 Hover card

On hover (or keyboard focus), a card appears with source metadata.

Contents.

- Source title (bold).
- Authors, comma-separated.
- Page reference (e.g., “p. 42” or “pp. 42–44”).
- Section path (e.g., “Chapter 3 › 3.2 Beta Reduction”).
- Excerpt: the first ~300 characters of the chunk text, with an ellipsis if truncated.

Behavior. Card appears after a 300ms hover delay (avoids accidental appearances on cursor pass-through). Card dismisses on mouseleave with a 100ms delay (avoids flicker on brief cursor movement). Card is keyboard-accessible: focused citation shows the card; Escape dismisses.

Full-passage view. A “Read full passage” link at the bottom of the card opens a modal with the complete chunk text, formatted with the source’s structural context.

10.3 Retired sources

Per retrieval §12, a citation whose source has been retired renders differently:

- Superscript number rendered in a muted color, no underline.
- Hover card shows a note: “This source is no longer available. The citation is preserved for historical reference.”
- No excerpt shown.

11. Practice and comprehension UI

Two related but distinct interaction patterns: comprehension checks (mid-lecture, per subsystem 2 §10) and lab practice (in the bench surface, per §6.3).

11.1 Comprehension check

Renders as an inline card within the lecture stream.

Layout.

Quick check

[question text]

Your answer...

[Submit answer]

First attempt. Learner types answer, submits. Loading state (“Checking...”), then verdict appears below the answer:

- **Correct:** green icon + text. “Nice. [feedback].” Lecture continues after a 1s pause.
- **Partial:** amber icon + text. “Close. [feedback].” Hint appears; second attempt allowed.
- **Incorrect:** red icon + text. “Not quite. [feedback].” Hint appears; second attempt allowed.

Second attempt. Same flow. Correct: continue. Incorrect or partial: model answer revealed with a “Here’s what a full answer looks like:” framing. Lecture continues.

Never punitive. Nothing about the UI treats a failed check as a failure. Mastery signal is recorded to the backend (as `mastery_events.kind = 'lecture_check_incorrect'`), but the surface framing is “we noticed this needs another pass” not “you got it wrong.”

11.2 Lab practice

Renders on the bench surface (§6.3).

Problem card. Left column. Contains problem statement, any setup, and constraints. May include an “expected time” hint from the Lecturer’s metadata.

Workspace. Below the problem or in a right column depending on viewport. For text answers: a textarea with sensible defaults. For code (deferred): Monaco editor. For math (deferred): a KaTeX-live editor.

Submit and feedback. Submit button below the workspace. On submit: loading state, then verdict in the right column (or below on narrow viewports). Rubric-based feedback for assessment attempts; single-verdict for practice.

Hints. Collapsed by default in a “Show hint” disclosure. Expanding logs the event but doesn’t penalize. Hints appear one at a time — the second hint requires expanding a “Show another hint” disclosure below the first.

11.3 Assessment attempts

Deferred for MVP (summative assessment engine is one of the four deferred modules). Architectural note: assessment UI would extend the lab surface with a stricter mode — no hints, no early feedback, all rubric criteria presented up front, single submission per criterion.

12. Confusion journal UI

Detailed above in §6.4. Two additional concerns worth naming here:

12.1 Learner authorship

The journal entry has three text fields (per data layer §6.7): `summary` (learner- or Tutor-authored), `hypothesis` (Tracker's inference), `learner_note` (learner-only). The UI enforces the difference:

- `summary` is editable by the learner but shows a subtle “written by [you|the tutor]” attribution.
- `hypothesis` is read-only for the learner. A “why this was flagged” tooltip explains the framing.
- `learner_note` is the learner's own space, always editable, no attribution.

Autosave on `learner_note` with a 2-second debounce, with a “Saved” indicator for reassurance.

12.2 Marking resolved

The “Mark resolved” action is available on any open or partial entry. On click:

1. Confirm dialog: “Mark this resolved? You can reopen it later if the confusion returns.”
2. On confirm: optimistic UI update (entry moves to the resolved list), POST to backend, rollback + toast on error.

Resolving is not a triumphant animation; it's a subtle transition. The framing: “this is no longer open” not “you've beaten it.” Consistent with §3's principle about not gamifying learning.

13. Accessibility (WCAG 2.2 Level AA)

This section is not a summary of WCAG requirements — those are documented elsewhere. It is a spec of how each requirement lands in Studium's frontend, with the specific components and patterns that implement it.

13.1 Perceivable

Text alternatives (WCAG 1.1.1). Every image, icon, and non-text control has a text equivalent. Lucide icons within buttons have accompanying text labels or `aria-label`.

Decorative icons have `aria-hidden="true"`. Charts and graphs (mastery summary on the desk) have a text data table equivalent, toggleable from the chart.

Adaptable (WCAG 1.3.1). All UI is built with semantic HTML. Headings use `<h1>`–`<h6>` in proper hierarchy (one `<h1>` per surface, `<h2>` for major sections, `<h3>` for subsections, no skipping levels). Landmarks (`<main>`, `<nav>`, `<aside>`, `<footer>`) demarcate regions. Lists use `<ul>` and `<ol>`. Form fields have programmatic `<label>` associations.

Distinguishable (WCAG 1.4.3, 1.4.11). Text has minimum 4.5:1 contrast against its background; large text (18pt+ or 14pt+ bold) minimum 3:1. UI components and graphical objects have minimum 3:1 contrast against adjacent colors. Both are asserted by axe-core in CI.

Reflow (WCAG 1.4.10). Content reflows at 320px width without horizontal scrolling except for content that requires 2D layout (code blocks with long lines, math with long expressions).

Use of color (WCAG 1.4.1). No information is conveyed by color alone. Verdict indicators (correct/partial/incorrect) always have both a color and an icon plus text. Error states have both a color and an icon plus copy.

Reduced motion (WCAG 2.3.3). `prefers-reduced-motion`: reduce respected everywhere. All animations disable or become near-instantaneous. No parallax, no floating cards, no auto-playing content.

13.2 Operable

Keyboard (WCAG 2.1.1). Every interactive element is reachable and operable by keyboard. Tab order matches visual order. No keyboard traps except intentional ones (modal dialogs, per WCAG 2.1.2). Global shortcuts documented in a ? help overlay accessible from any surface.

Focus visible (WCAG 2.4.7). Focus indicators are clearly visible. Default browser outline is preserved or replaced with a more visible custom outline (never `outline: none` without replacement). Radix's focus-visible primitives are used for interactive components.

Focus not obscured (WCAG 2.4.11). New in WCAG 2.2. Focused elements are not entirely hidden by sticky headers, floating action groups, or other overlays. The bottom-right floating action group in the session surface accounts for this: focused elements near the bottom of the viewport scroll into view above the floating group.

Timing (WCAG 2.2.1). Session timers do not close a session automatically without warning. The idle timeout at `target_duration_minutes + 15` shows a warning at 5 minutes remaining and a confirmation prompt at 1 minute; learner can extend.

Interruptions (WCAG 2.2.2). Auto-updating content (streaming) can be paused (via interrupt), and non-essential animations respect `prefers-reduced-motion`.

13.3 Understandable

Language (WCAG 3.1.1). Every page has lang on the `<html>` element (default en-CA, per user profile).

Predictable (WCAG 3.2.1, 3.2.2). Focus changes do not trigger navigation. Form field changes do not trigger navigation. The command palette is the exception: hitting Enter on a primitive triggers its invocation, but this is expected behavior for command palettes and is documented in the help overlay.

Consistent identification (WCAG 3.2.4). The same icon means the same thing throughout. Hand-raise is always the “interrupt” icon; the same green checkmark is always “correct.” Icon meanings documented in `components/ui/icons.tsx`.

Input assistance (WCAG 3.3.1, 3.3.2). Form errors are announced via aria-live and shown adjacent to the field. Labels describe expected input format where non-obvious.

Consistent help (WCAG 3.2.6). New in WCAG 2.2. Help resources (the ? overlay, contact link, keyboard shortcut reference) appear in the same location on every surface.

13.4 Robust

Parsing. No inline styles, no legacy HTML, no ARIA without corresponding functionality. All React output validates as HTML5.

Name, role, value (WCAG 4.1.2). All custom components (via Radix or hand-built) expose proper accessibility trees. Buttons have accessible names; toggles have accessible states; regions have accessible descriptions.

Status messages (WCAG 4.1.3). Announcements via aria-live — polite for stream text arrivals, assertive for interrupt state changes, errors, and completions. A single announcement region at the top of the app receives these; component-level aria-live is avoided to prevent overlap.

13.5 Verification

axe-core runs against every component in the unit test suite. Any WCAG 2.2 AA violation fails the test. The failing rule and its remediation are surfaced in the test output.

Playwright E2E tests include a “keyboard-only navigation” pass through the primary surfaces: desk → session start → classroom → primitive palette → journal. Any surface that cannot complete this pass fails.

Manual verification: monthly screen-reader test using VoiceOver on macOS Safari, at minimum. Additional testing with NVDA on Windows Firefox as time permits. Neither is scriptable but both are essential — automated tools catch structural violations, screen-reader testing catches interaction quality.

14. State management

14.1 Server state (TanStack Query)

Every server data access flows through a TanStack Query hook. Query keys are typed. Invalidation is explicit.

Example hook.

```
export function useOpenJournalEntries(learnerSubjectId: UUID) {
  return useQuery({
    queryKey: ['journal', learnerSubjectId, { status: ['open', 'partial'] }],
    queryFn: () => fetchJournalEntries(learnerSubjectId, { status: ['open', 'partial'] }),
    staleTime: 30_000,
  });
}
```

Mutation with optimistic update.

```
export function useResolveJournalEntry() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (entryId: UUID) => resolveJournalEntry(entryId),
    onMutate: async (entryId) => {
      await queryClient.cancelQueries({ queryKey: ['journal'] });
      const previous = queryClient.getQueriesData({ queryKey: ['journal'] });
      queryClient.setQueriesData({ queryKey: ['journal'] }, (old:
JournalEntry[]) =>
        old.map(e => e.id === entryId ? { ...e, status: 'resolved' } : e)
      );
      return { previous };
    },
    onError: (_, __, context) => {
      context?.previous.forEach(([key, data]) =>
queryClient.setQueryData(key, data));
      toast.error('Could not mark resolved. Please try again.');
```

14.2 Client state (Zustand)

Global client state has three stores:

useSessionStore. Current session state — session ID, mode, focus concept, stream buffer, interrupt state, current turn index. Reset on session close.

useUIStore. UI preferences that shouldn't require a round trip — dark mode, font size, cost display toggle, auto-scroll preference. Persisted to localStorage.

useCommandPaletteStore. Command palette visibility, filter state, focused option. Ephemeral.

Component-local state (useState) for concerns that don't leak out of a component: form input value, expanded/collapsed disclosure, hover state.

14.3 URL state

URL is the source of truth for navigation. Query params for filter state:

- /journal?status=open,partial&subject=lambda-calculus
- /sessions/[session_id]?mode=lecture

TanStack Query invalidates queries when URL filter params change so the data refetches with the new filter.

15. Error handling and degradation UI

15.1 Error boundaries

Every route has an `error.tsx` that catches render errors. The error boundary shows a calm, honest message (“Something went wrong loading this page”) with two actions: “Try again” (reset the error boundary) and “Return to the desk” (navigate home).

Errors are logged to observability (Sentry, spec'd in Infrastructure) with the user ID (if authenticated) and the route.

15.2 API errors

Every API call has a defined error path. HTTP 4xx: shows a message specific to the status (“Not found”, “Access denied”, “Session not started”). HTTP 5xx: shows a generic “The server is having trouble right now” message with retry.

Budget cap errors (from subsystem 2 §19) have specific copy that names the cap and its reset time in the learner's timezone. Never a raw dollar figure without context.

Rate limit errors (from the model provider) have specific copy: “The system is under high load. Please try again in a moment.” No specific numbers; the intent is calm, not diagnostic.

15.3 Network drops

If the SSE connection drops during a stream, the reconnect logic (§8.1) attempts to resume. If reconnection fails after three attempts, the classroom shows: “Lost connection to the tutor. Trying to reconnect...” with a manual “Try again” button. On persistent failure: “We can't reach the tutor right now. Your progress is saved. Try again in a few minutes.”

15.4 Degradation copy library

All degradation copy lives in `lib/copy/degradation.ts`, matching the subsystem 2 §21 pattern. This ensures consistency, makes tone-tuning a single change, and prepares for localization later.

16. Aesthetic and typography

The product plan §9 named the aesthetic direction: a well-kept study, publisher book design, journal typography, library reading room. This section makes it concrete.

16.1 Type

Body serif. Source Serif 4 (or Charter as a system-safe alternative). Loaded via `next/font/google`. Used for all reading content: lecture text, tutorial exchanges, journal entries, summaries.

UI sans. Inter (or system-ui as fallback). Used for buttons, labels, form fields, chrome, navigation, and any content-adjacent metadata.

Monospace. JetBrains Mono. Used for code blocks and inline code.

Scale. Modular scale with base 16px, ratio 1.2:

- 12px / 14px / 16px (body) / 20px / 24px / 28px / 34px / 41px

Line height 1.55 for body prose, 1.25 for headings. Measure (line length) capped at ~65 characters for body text.

16.2 Palette

Restrained neutral palette with one accent. Values as HSL for easy dark-mode inversion.

Light mode.

- Background: `hsl(40, 30%, 97%)` — a warm off-white, not pure white
- Surface: `hsl(40, 20%, 94%)` — slightly darker off-white for cards
- Text primary: `hsl(220, 15%, 18%)` — deep ink, not pure black
- Text secondary: `hsl(220, 10%, 42%)` — muted for metadata
- Border: `hsl(40, 15%, 82%)` — subtle
- Accent: `hsl(200, 45%, 40%)` — a muted teal, used sparingly for links, focus, active states
- Correct: `hsl(150, 40%, 38%)` — muted green
- Attention: `hsl(30, 60%, 45%)` — muted amber
- Concern: `hsl(0, 45%, 45%)` — muted red

Dark mode. Inversions of the above with adjusted lightness to maintain 4.5:1 contrast.

No gradients. No shadows beyond subtle single-layer elevation. No glassmorphism. The visual language is flat, quiet, and confident.

16.3 Spacing

Tailwind's default scale, with named tokens for common uses:

- `space-tight` (4px) — within components
- `space-normal` (16px) — between related elements
- `space-loose` (32px) — between sections
- `space-generous` (64px) — between major regions

16.4 Motion

Motion serves function, not decoration.

- **Transitions:** 150ms for hover states, 200ms for content transitions, 300ms for surface changes. Easing: `cubic-bezier(0.4, 0.0, 0.2, 1)`.
- **No entrance animations.** Content appears; it doesn't slide in, fade in, or scale.
- **Exceptions.** Toast notifications slide in from the top. Modal dialogs fade in with the backdrop.
- **Reduced motion.** Everything above collapses to instantaneous under `prefers-reduced-motion: reduce`.

16.5 Copy tone

Copy is direct, calm, and treats the learner as an adult undertaking serious work.

Yes. "Ready when you are." "Not quite — try once more?" "Session complete. Ready for the next?"

No. "Awesome job!!!" "You're crushing it!" "Let's smash this challenge!" "🔥 🔥 🔥 3-day streak!"

The full copy library lives in `lib/copy/` organized by surface. A single change to tone tunes the whole product.

16.6 Iconography

Lucide React exclusively. 1.5px stroke weight, 20px default size, 16px in dense contexts. Icons pair with text labels or `aria-label` per §13's requirements.

17. Testing strategy

Same tiered structure as prior subsystems.

Tier 1 — Offline (component and unit tests).

- Component rendering: every surface renders with a minimal fixture without errors.
- Accessibility: axe-core assertions on every component. Zero WCAG 2.2 AA violations.
- State management: Zustand stores behave correctly under sequence of actions.

- Zod schema validation: API response fixtures validate cleanly; malformed fixtures fail cleanly.
- Streaming parser: given SSE event fixtures, chunks dispatch correctly.
- Sentence-boundary detection on the client side (for interrupt UI): given a stream of tokens, detects sentence completion correctly.
- Citation marker parser: given rendered text with markers, produces correct component tree.
- Coverage guard on the accessibility test suite: if no components are exercised, the suite fails rather than passing vacuously.

Tier 2 — Online (E2E against a live backend, no LLM calls).

- Session lifecycle: create session with a mocked-LLM backend, verify the classroom mounts and receives the mock stream.
- Interruption round-trip: mock a stream, trigger interrupt, verify state transitions and UI updates.
- Journal CRUD: create, view, edit, resolve entries; verify optimistic updates and rollback.
- Auth flow: sign in with a test user, land on the desk.
- Keyboard-only navigation: complete the primary flows using only keyboard.

Tier 3 — Paid (E2E with real Anthropic and Voyage APIs).

- Real session end-to-end: open a session on the seeded lambda calculus subject, receive real Lecturer output, invoke `explain_differently`, verify the new stance streams and renders.
- Interrupt during real stream: send interrupt mid-lecture, verify the actual sentence-boundary detection works, verify the Tutor's response streams and renders.
- Citation resolution: hover over a [P3] marker in real output, verify the hover card populates with real source metadata.

Fixtures. Playwright test fixtures include an authenticated user, a seeded lambda calculus subject, and a session in each mode. Fixtures reset between tests.

CI wiring. Same pattern as prior subsystems: Tier 1 on every push, Tier 2 on main and yannis, Tier 3 on-demand with `STUDIUM_RUN_PAID_TESTS=1`. A `.github/workflows/frontend.yml` per the dev's principle of one workflow per subsystem.

18. Version history

v1.0 — 18 August 2026. Initial specification. Written against data layer v1.1, agent runtime v1.0, and retrieval v1.0. Locks the Next.js + Radix + Tailwind stack, the five MVP surfaces, the streaming and interruption UX, the citation rendering approach, the tutorial primitives UI, the WCAG 2.2 AA compliance requirements, and the aesthetic system. Explicit forward references to Content Ingestion (§6.4 mentions ingestion of learner-uploaded content), Evaluation (§17's regression testing on prompts and rendered output), and Infrastructure (§4's deployment target, §15's observability integration).

Anticipated v1.1 candidates.

- The concept graph map surface when the concept graph module ships. Extends the classroom (right sidebar) and adds a full /subjects/[slug]/map route.
- The review cycle UI when spaced repetition ships. Extends the desk (review queue prominence) and adds a /review route.
- The multi-voice library UI when it ships. Extends the classroom (stance switcher near concept name).
- The summative assessment UI when it ships. Extends the bench with a stricter mode.
- Mobile support if MVP demand justifies it. Would require a separate mobile spec, not just responsive refinement.

19. Forward references and open questions

Content Ingestion spec (subsystem 5). Where the corpus is authored. This subsystem consumes source metadata via API but does not author it. If learner-upload becomes a MVP feature (currently deferred), an upload UI would live here.

Evaluation spec (subsystem 6). Golden datasets for rendering regression: given fixed backend outputs, verify the frontend renders them consistently across releases. The hooks for these tests exist in Tier 2 fixtures.

Infrastructure spec (subsystem 7). Fly.io deployment configuration for the Next.js app. CDN configuration if static assets outgrow the origin. Analytics wiring (which analytics tool, what events, what privacy posture). Error observability (Sentry configuration, sampling rate). Session replay if debugging warrants (LogRocket or similar; considered but deferred to when actually needed).

Open questions requiring build-time answers.

1. **Sentence-boundary detection on the client.** The subsystem 2 spec has the server doing sentence-boundary detection for interruption. The client also needs some form of sentence detection to render the “still generating” indicator at the correct boundary while streaming. Whether the same algorithm can be used client-side (JavaScript regex + heuristics) or whether we lean on server-provided boundary markers is a build-time question.
2. **Command palette default vs discoverability.** If learners rarely discover the command palette, the primitives are effectively invisible. Consider a first-run tutorial or persistent hint. Measurable from primitive invocation rates once real learners are using the product.
3. **The reading vs interaction viewport split.** The current design gives interaction affordances (buttons, palette trigger) permanent screen real estate at the bottom-right. If real users find this distracting during long reading passages, a “focus mode” that hides chrome might be needed. Anticipated but not spec'd.

4. **Dark mode default.** Some learners will study late at night. The `prefers-color-scheme` system preference is respected by default. Whether the app should have a preference override or trust the system is a small UX question that will land where the reviewer prefers.
 5. **Accessibility for math and code.** KaTeX has ARIA rendering options for screen readers; Shiki has less. What screen-reader users experience with math-heavy or code-heavy content is a real question that needs actual testing with actual users; deferred to post-MVP unless a screen-reader-dependent learner shows up in the first users.
-

End of specification

This document defines the frontend for Studium in full. Five surfaces, one streaming architecture, eight tutorial primitives with three access paths, one aesthetic system, and WCAG 2.2 Level AA compliance throughout. A senior engineer with data layer v1.1, agent runtime v1.0, retrieval v1.0, and this document can build a working web frontend that consumes real agent streams and produces the four MVP interaction modes end-to-end for a serious learner working through lambda calculus.

Next in sequence: **Subsystem 5 — Content Authoring and Ingestion**, which specifies the pipeline from uploaded PDF to indexed chunks to concept-graph-linked passages, and the authoring tools for the concept graph itself, rubric criteria, and per-artifact metadata. It is written against all four prior specs.